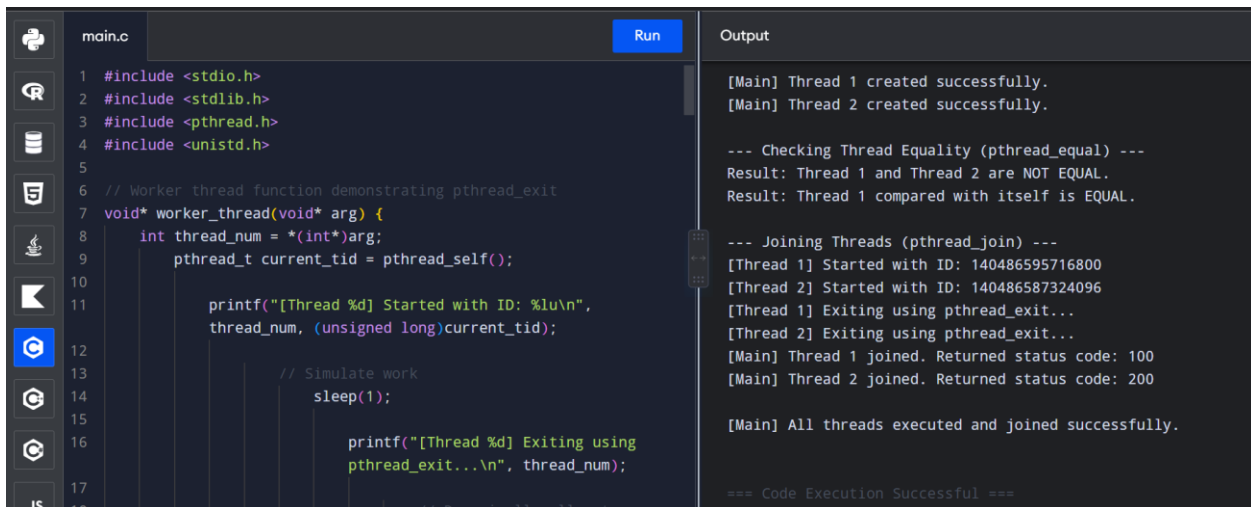


EXPERIMENT 40:

40. Write C programs to demonstrate the following thread-related concepts: (i) create, (ii) join, (iii) equal, and (iv) exit.



The screenshot shows a C program in a code editor with a dark theme. The code is in a file named 'main.c' and includes headers for stdio, stdlib, pthread, and unistd. It defines a worker thread function 'worker_thread' that takes a pointer to an integer 'thread_num' as an argument. Inside the function, it prints the thread's ID using 'pthread_self()' and 'pthread_exit()'. The main function creates two threads, 'Thread 1' and 'Thread 2', using 'pthread_create()'. It then checks their equality using 'pthread_equal()', joins them using 'pthread_join()', and prints the status codes. The output window on the right shows the execution results, confirming that the threads were created successfully, their IDs were compared, and they were joined successfully.

```
main.c
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <pthread.h>
4 #include <unistd.h>
5
6 // Worker thread function demonstrating pthread_exit
7 void* worker_thread(void* arg) {
8     int thread_num = *(int*)arg;
9     pthread_t current_tid = pthread_self();
10
11     printf("[Thread %d] Started with ID: %lu\n",
12           thread_num, (unsigned long)current_tid);
13
14     // Simulate work
15     sleep(1);
16
17     printf("[Thread %d] Exiting using
18           pthread_exit...\n", thread_num);
19
20     pthread_exit(NULL);
21 }
22
23 int main() {
24     pthread_t tid1, tid2;
25     int status1, status2;
26
27     // Create Thread 1
28     pthread_create(&tid1, NULL, worker_thread, (void*)1);
29
30     // Create Thread 2
31     pthread_create(&tid2, NULL, worker_thread, (void*)2);
32
33     // Check Thread Equality (pthread_equal)
34     pthread_t tid1_self = pthread_self();
35     pthread_t tid2_self = pthread_self();
36     pthread_t tid1_equal = pthread_equal(tid1_self, tid2_self);
37
38     // Joining Threads (pthread_join)
39     pthread_join(tid1, &status1);
40     pthread_join(tid2, &status2);
41
42     printf("Main Thread 1 joined. Returned status code: %d\n", status1);
43     printf("Main Thread 2 joined. Returned status code: %d\n", status2);
44
45     printf("All threads executed and joined successfully.\n");
46     return 0;
47 }
```

Output

```
[Main] Thread 1 created successfully.
[Main] Thread 2 created successfully.

--- Checking Thread Equality (pthread_equal) ---
Result: Thread 1 and Thread 2 are NOT EQUAL.
Result: Thread 1 compared with itself is EQUAL.

--- Joining Threads (pthread_join) ---
[Thread 1] Started with ID: 140486595716800
[Thread 2] Started with ID: 140486587324096
[Thread 1] Exiting using pthread_exit...
[Thread 2] Exiting using pthread_exit...
[Main] Thread 1 joined. Returned status code: 100
[Main] Thread 2 joined. Returned status code: 200

[Main] All threads executed and joined successfully.

=== Code Execution Successful ===
```

PSEUDO CODE :

- Define a thread function `worker_thread` that accepts an argument (ID):
- Print the current thread's ID using `pthread_self()`.
- Exit gracefully using `pthread_exit()`, passing a return status pointer back to the main thread.
- In the main function:
- Declare two thread variables (`tid1`, `tid2`).
- Create `tid1` and `tid2` using `pthread_create()`, passing respective arguments.
- Compare `tid1` and `tid2` using `pthread_equal(tid1, tid2)` to demonstrate thread ID comparison. Also compare `tid1` with itself.
- Call `pthread_join(tid1, &status)` to block the main thread until `tid1` completes and retrieve its exit status.
- Call `pthread_join(tid2, &status)` to wait for `tid2`.
- Print confirmation that all threads completed and terminate the main program.

CODE:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <pthread.h>
```

```
#include <unistd.h>
```

```
// Worker thread
```

```
function demonstrating
```

```
pthread_exit
```

```
void*
```

```
worker_thread(void*
```

```
arg) {
```

```
    int thread_num =
```

```
    *(int*)arg;
```

```
    pthread_t current_tid
```

```
= pthread_self();
```

```
    printf("[Thread %d]
```

```
Started with ID: %lu\n",
```

```
thread_num, (unsigned
```

```
long)current_tid);
```

```
// Simulate work
```

```
sleep(1);
```

```
printf("[Thread %d]
```

Exiting using

```
pthread_exit...\n",
```

```
thread_num);
```

```
// Dynamically
```

allocate return status to

pass back to

```
pthread_join
```

```
int *status =
```

```
malloc(sizeof(int));
```

```
*status = thread_num
```

```
* 100;
```

```
// (iv) exit: Terminate
```

calling thread

```
pthread_exit((void*)stat
```

```
us);
```

```
}
```

```
int main() {  
  
    pthread_t tid1, tid2;  
  
    int id1 = 1, id2 = 2;  
  
    void *status1,  
    *status2;  
  
  
    printf("---  
Demonstrating Thread  
Concepts ---\n\n");  
  
  
    // (i) create: Creating  
Thread 1 and Thread 2  
  
    if  
(pthread_create(&tid1,  
NULL, worker_thread,  
&id1) != 0) {  
  
        perror("Failed to  
create thread 1");  
  
        return 1;  
    }  
}
```

```
}

printf("[Main] Thread
1 created
successfully.\n");
```

```
if
(pthread_create(&tid2,
NULL, worker_thread,
&id2) != 0) {
```

```
    perror("Failed to
create thread 2");
```

```
    return 1;
```

```
}
```

```
printf("[Main] Thread
2 created
successfully.\n\n");
```

```
// (iii) equal:
```

```
Comparing Thread IDs
```

```
using pthread_equal
```

```
    printf("--- Checking  
Thread Equality  
(pthread_equal) ---\n");
```

```
    if  
(pthread_equal(tid1,  
tid2)) {
```

```
        printf("Result:  
Thread 1 and Thread 2  
are EQUAL.\n");
```

```
    } else {
```

```
        printf("Result:  
Thread 1 and Thread 2  
are NOT EQUAL.\n");
```

```
    }
```

```
    if  
(pthread_equal(tid1,  
tid1)) {
```

```
        printf("Result:  
Thread 1 compared with  
itself is EQUAL.\n\n");
```

```
}
```

```
// (ii) join: Main waits  
for threads to complete  
and retrieves exit status
```

```
printf("--- Joining  
Threads (pthread_join) -  
--\n");
```

```
pthread_join(tid1,  
&status1);
```

```
printf("[Main] Thread  
1 joined. Returned  
status code: %d\n",  
*(int*)status1);
```

```
free(status1);
```

```
pthread_join(tid2,  
&status2);
```

```
printf("[Main] Thread  
2 joined. Returned  
status code: %d\n",
```

```
*(int*)status2);
```

```
free(status2);
```

```
printf("\n[Main] All  
threads executed and  
joined successfully.\n");
```

```
return 0;
```

```
}
```

INPUT :

(No direct user input
required; parameters are
initialized within
main())

OUTPUT:

--- Demonstrating
Thread Concepts ---

[Main] Thread 1 created

successfully.

[Main] Thread 2 created

successfully.

--- Checking Thread

Equality

(pthread_equal) ---

Result: Thread 1 and

Thread 2 are NOT

EQUAL.

Result: Thread 1

compared with itself is

EQUAL.

--- Joining Threads

(pthread_join) ---

[Thread 1] Started with

ID: 140445241775872

[Thread 2] Started with

ID: 140445233383168

[Thread 1] Exiting using
pthread_exit...

[Thread 2] Exiting using
pthread_exit...

[Main] Thread 1 joined.

Returned status code:

100

[Main] Thread 2 joined.

Returned status code:

200

[Main] All threads
executed and joined
successfully.